Laboratory reportof Nanjing University of Information Science and TechnologyExperiment

Title Lab9 Date 2026.6.11 Class 24AC
Name ZhuangZhong ID 202483910032

1. Objectives

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical "disks" are destroyed.

2. Environment Preparation

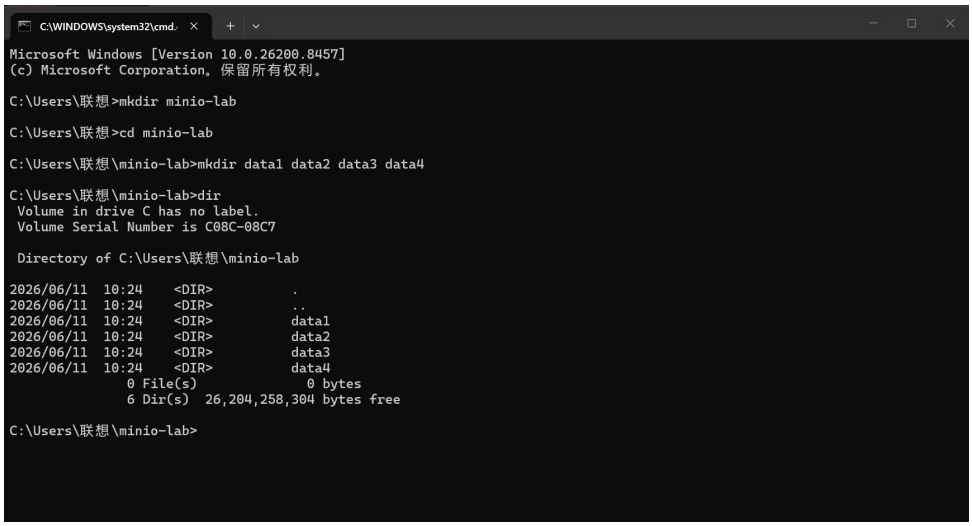
In this experiment, students will simulate a distributed cloud object storage system similar to Amazon S3 or Alibaba Cloud OSS using:

MinIO – open-source S3-compatible object storage
Docker Compose – container orchestration

3. Experiment Implementation and Results

3.1Create a Dedicated Directory

Create a directory for this lab and navigate into it.



3.2: Create Storage Directories

Create four independent storage directories to simulate four storage disks.
mkdir data1 data2 data3 data4
Verify with dir – you should see the four directories listed.

3.3: Create docker-compose.yml

Create a file named docker-compose.yml with the following content:

```
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    command: server /data{1...4} --console-address ":9001"

  mc:
    image: minio/mc
    container_name: cloud-mc
    depends_on:
      - minio
    entrypoint: /bin/sh
    tty: true
```

3. 4: Explain the docker-compose.yml File

Component	Meaning
image	Docker image used to create the container
container_name	Name assigned to the running container
volumes	Mounts host directories into the container (persistent storage)
ports	Maps host ports to container ports (9000: API, 9001: Console)
environment	Sets environment variables (root user credentials)
command	Starts MinIO distributed storage service with 4 disks
depends_on	Ensures mc container starts after minio

3.5: Launch the Distributed Cluster

Start all services using Docker Compose:

bash

```
docker compose up -d
```

```
C:\Users\联想\minio-lab>notepad docker-compose.yml

C:\Users\联想\minio-lab>docker compose up -d
[+] up 19/19
 ✔ Image minio/mc              Pulled                                    137.1s
 ✔ Image quay.io/minio/minio   Pulled                                    12.2ss
 ✔ Network minio-lab_default   Created                                   0.1s
 ✔ Container cloud-minio       Started                                   0.6s
 ✔ Container cloud-mc          Started                                   0.7s

C:\Users\联想\minio-lab>
```

Figure 4 – Pulling images and starting containers with docker compose up -d.

Verify running containers:

bash

```
docker ps
```

You should see both cloud-minio and cloud-mc containers running.

3.6: Configure MinIO Client Alias
Configure the MinIO client (mc) to connect to the MinIO server:

bash
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123

```
C:\Users\联想\minio-lab>docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added `myminio` successfully.

C:\Users\联想\minio-lab>
```

Step 7: Create a Bucket
Create a cloud storage bucket named my-cloud-storage:

bash
docker exec cloud-mc mc mb myminio/my-cloud-storage

```
C:\Users\联想\minio-lab>docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully `myminio/my-cloud-storage`.

C:\Users\联想\minio-lab>
```

3. 8: Create and Upload a Large Test File
Create a Large Test File (20 MB)
Verify the File Size

```
C:\Users\联想\minio-lab>dir bigfile.bin
 Volume in drive C has no label.
 Volume Serial Number is C08C-08C7

 Directory of C:\Users\联想\minio-lab

2026/06/11  10:29        20,971,520 bigfile.bin
               1 File(s)     20,971,520 bytes
               0 Dir(s)  25,646,333,952 bytes free

C:\Users\联想\minio-lab>docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin

C:\Users\联想\minio-lab>docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
┌──────────┬─────────────┬──────────┬────────────┐
│ Total    │ Transferred │ Duration │ Speed      │
│ 20.00 MiB │ 20.00 MiB   │ 00m00s   │ 49.78 MiB/s │
└──────────┴─────────────┴──────────┴────────────┘

C:\Users\联想\minio-lab>
```

Figure 7 – The 20MB test file created.

8.3 Copy the File into the mc Container
bash
docker cp bigfile.bin cloud-mc:/bigfile.bin

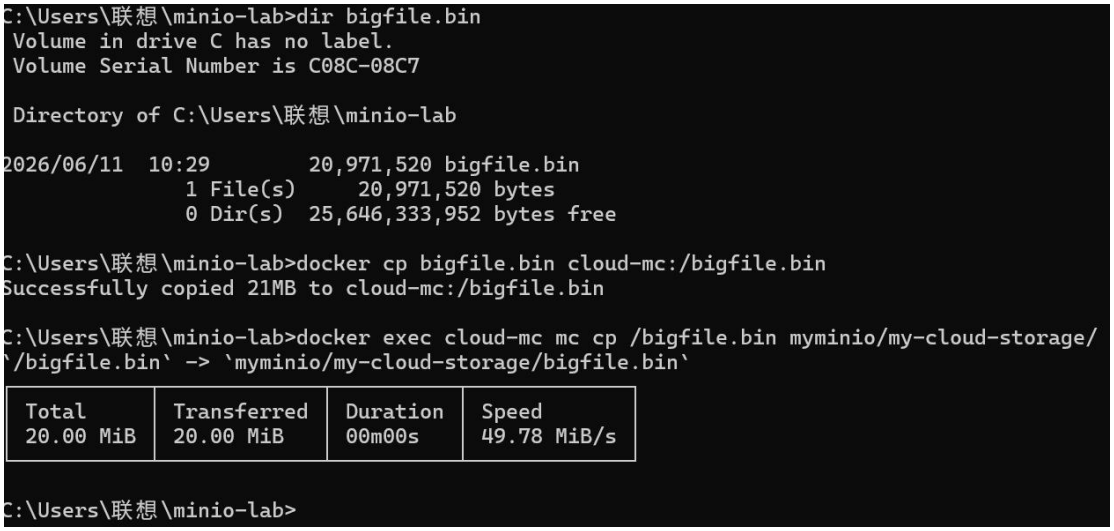


Figure 8 – File copied into the mc container.

8.4 Upload the File to Cloud Storage
bash
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
Figure 9 – File uploaded to MinIO bucket with speed and duration shown.
Step 9: Verify Object Upload
List all objects in the bucket:
bash
docker exec cloud-mc mc ls myminio/my-cloud-storage/
You should see bigfile.bin listed.
Step 10: Observe Backend Storage Fragmentation
Observe how MinIO distributes data across multiple storage disks:

bash
dir data1
dir data2
dir data3
dir data4

```
2026/06/11  10:28    <DIR>          .
2026/06/11  10:29    <DIR>          ..
2026/06/11  10:27    <DIR>          .minio.sys
2026/06/11  10:29    <DIR>          my-cloud-storage
               0 File(s)              0 bytes
               4 Dir(s)  25,508,438,016 bytes free

C:\Users\联想\minio-lab>dir data2
 Volume in drive C has no label.
 Volume Serial Number is C08C-08C7

 Directory of C:\Users\联想\minio-lab\data2

2026/06/11  10:28    <DIR>          .
2026/06/11  10:29    <DIR>          ..
2026/06/11  10:27    <DIR>          .minio.sys
2026/06/11  10:29    <DIR>          my-cloud-storage
               0 File(s)              0 bytes
               4 Dir(s)  25,504,104,448 bytes free

C:\Users\联想\minio-lab>dir data3
 Volume in drive C has no label.
 Volume Serial Number is C08C-08C7

 Directory of C:\Users\联想\minio-lab\data3

2026/06/11  10:28    <DIR>          .
2026/06/11  10:29    <DIR>          ..
2026/06/11  10:27    <DIR>          .minio.sys
2026/06/11  10:29    <DIR>          my-cloud-storage
               0 File(s)              0 bytes
               4 Dir(s)  25,492,336,640 bytes free

C:\Users\联想\minio-lab>dir data4
 Volume in drive C has no label.
 Volume Serial Number is C08C-08C7

 Directory of C:\Users\联想\minio-lab\data4

2026/06/11  10:28    <DIR>          .
2026/06/11  10:29    <DIR>          ..
2026/06/11  10:27    <DIR>          .minio.sys
2026/06/11  10:29    <DIR>          my-cloud-storage
               0 File(s)              0 bytes
               4 Dir(s)  25,477,611,520 bytes free

C:\Users\联想\minio-lab>
```

Figure 10 – Storage fragmentation across data1–data4. Each directory contains .minio.sys and my-cloud-storage folders.

Observations:

Each data directory contains .minio.sys (system metadata) and my-cloud-storage (bucket data)

The object metadata (xl.meta) is distributed across all disks

This demonstrates MinIO's erasure coding mechanism

Step 11: Simulate Disk Failure
11.1 Stop the Cluster
bash

docker compose down

```
C:\Users\联想\minio-lab>docker compose down
[+] down 3/3
 ✔Container cloud-mc         Removed
 ✔Container cloud-minio      Removed
 ✔Network minio-lab_default  Removed

C:\Users\联想\minio-lab>rmdir /s /q data3

C:\Users\联想\minio-lab>mkdir data3

C:\Users\联想\minio-lab>docker compose up -d
[+] up 3/3
 ✔Network minio-lab_default  Created
 ✔Container cloud-minio      Started
 ✔Container cloud-mc         Started

C:\Users\联想\minio-lab>
```

Figure 11 – Cluster stopped.

11.2 Delete One Storage Disk
Delete data3 and recreate it (simulating disk failure):

bash
rmdir /s /q data3
mkdir data3
11.3 Restart the Cluster
bash
docker compose up -d

```
C:\Users\联想\minio-lab>docker compose down
[+] down 3/3
 ✔Container cloud-mc         Removed
 ✔Container cloud-minio      Removed
 ✔Network minio-lab_default  Removed

C:\Users\联想\minio-lab>rmdir /s /q data3

C:\Users\联想\minio-lab>mkdir data3

C:\Users\联想\minio-lab>docker compose up -d
[+] up 3/3
 ✔Network minio-lab_default  Created
 ✔Container cloud-minio      Started
 ✔Container cloud-mc         Started

C:\Users\联想\minio-lab>
```

Why is data still available?
MinIO uses erasure coding. With 4 disks, data is split into fragments and distributed.

Even if one disk is lost, the remaining fragments are sufficient to reconstruct the original file. This provides n/2 tolerance – in a 4-disk setup, up to 2 disks can fail without data loss.

Step 13: Use git to Save Your Changes
bash
git init
git add .
git commit -m "MinIO lab: distributed object storage"

Figure 14 – Git initialized and changes committed.

4. Issues and Solutions

Problem	Solution
dd command not found on Windows	Used fsutil file createnew to generate a 20MB test file
docker compose command not recognized	Ensured Docker Desktop is installed and running, then retried
Port 9000/9001 already in use	Stopped conflicting services or changed ports in docker-compose.yml
mca typo in command (mca instead of mc)	Corrected command to docker exec cloud-mc mc alias set ..

5. Conclusion
This lab successfully demonstrated the core principles of distributed object storage using MinIO. The following achievements were made:

4-node distributed cluster deployed using Docker Compose
Bucket created and 20MB test file uploaded
Backend storage fragmentation observed – data distributed across 4 disks
Disk failure simulated – data3 deleted and recreated
Fault tolerance verified – file still accessible after "disk loss"
Git version control used to track experiment artifacts

The experiment confirms that MinIO provides S3-compatible object storage with

built-in erasure coding and distributed redundancy, ensuring data durability and availability even when individual storage nodes fail. These principles form the foundation of production cloud storage systems like Amazon S3 and Alibaba Cloud OSS.